

ClimateProcessingEngine

A Comprehensive Python Framework for Climate Data
Processing

Amin Fazl Kazemi
`aminfazlkazemi@gmail.com`

Version 2.1
July 26, 2026

Contents

1	Introduction	8
1.1	Overview	8
1.1.1	Key Features	8
1.1.2	Target Audience	8
1.1.3	System Requirements	9
1.1.4	Dependencies	9
2	Architecture Overview	10
2.1	System Architecture	10
2.2	Core Components	11
2.2.1	Configuration Layer (<code>constants.py</code> + <code>config.yaml</code>)	11
2.2.2	I/O Pipeline (<code>io_pipeline/*.py</code>)	11
2.2.3	Numerical Engine (<code>numerical_engine/*.py</code>)	11
2.2.4	Result Pipeline (<code>result_pipeline/*.py</code>)	11
2.2.5	Monitoring (<code>monitoring/*.py</code>)	11
2.2.6	Orchestrator (<code>orchestrator/*.py</code>)	11
2.3	Data Flow	11
3	Installation and Setup	12
3.1	Installation	12
3.1.1	Clone the Repository	12
3.1.2	Create a Virtual Environment	12
3.1.3	Install Dependencies	12
3.2	Configuration	12
3.2.1	Configuration File Structure	12
3.2.2	Understanding Configuration Parameters	14
3.3	Preparing Input Data	14
3.3.1	Input Zarr Structure	14
3.3.2	Calendar File Format	15
4	Core Modules Reference	16
4.1	Configuration Module (<code>constants.py</code>)	16
4.1.1	Overview	16
4.1.2	Key Constants	16
4.1.3	Configuration Functions	17
4.2	Zarr Schema Module (<code>zarr_schema.py</code>)	17
4.2.1	Overview	17
4.2.2	Variable Definitions	17
4.2.3	Key Functions	18

4.3	Calendar Tables Module (calendar_tables.py)	18
4.3.1	Overview	18
4.3.2	Key Functions	18
4.4	Runtime Tables Module (runtime_tables.py)	19
4.4.1	Overview	19
4.4.2	Key Functions	19
4.5	I/O Pipeline Module (io_pipeline/*.py)	19
4.5.1	read_month_files.py	19
4.5.2	assemble_block.py	20
4.5.3	validate_block.py	20
4.6	Numerical Engine Module (numerical_engine/*.py)	21
4.6.1	window_engine.py	21
4.6.2	distributions.py	21
4.6.3	analyze_station.py	23
4.6.4	merge_results.py	24
4.7	Result Pipeline Module (result_pipeline/*.py)	24
4.7.1	validate_result.py	24
4.7.2	write_block.py	24
4.8	Orchestrator Module (orchestrator/*.py)	25
4.8.1	process_block.py	25
4.9	Monitoring Module (monitoring/*.py)	25
4.9.1	benchmark.py	25
4.9.2	checkpoint.py	25
4.9.3	logger.py	26
5	Data Contract	27
5.1	Dynamic Distribution Architecture	27
5.1.1	Key Concepts	27
5.1.2	Adding a New Distribution	27
5.1.3	Benefits	27
5.2	Load Phase Contract	28
5.2.1	Input	28
5.2.2	Output	28
5.3	Window Phase Contract	28
5.3.1	Input	28
5.3.2	Output	28
5.4	Analyze Phase Contract	28
5.4.1	Input	28
5.4.2	Output	29
5.5	Write Phase Contract	29
5.5.1	Input	29
5.5.2	Output	29
5.6	Constants	29
5.7	Error Types	29

6	Mathematical Background	30
6.1	Distributions	30
6.1.1	Normal Distribution	30
6.1.2	Skew-Normal Distribution	30
6.1.3	Bimodal Normal Distribution	30
6.1.4	Pearson Type III Distribution	30
6.2	Model Selection Criteria	30
6.2.1	AIC (Akaike Information Criterion)	30
6.2.2	AICc (Corrected AIC)	31
6.2.3	BIC (Bayesian Information Criterion)	31
6.3	Bimodality Metrics	31
6.3.1	Ashman's D	31
6.3.2	Bimodality Coefficient	31
6.3.3	Overlap Coefficient	31
6.4	Window-Based Estimation	31
7	Execution Flow	32
7.1	Main Function	32
7.2	Block Processing Flow	32
7.3	Recovery Flow	33
8	Input Data Format	34
8.1	Input Zarr Structure	34
8.2	Data Scaling	34
8.3	Missing Data	34
9	Output Format	35
9.1	Zarr Structure	35
9.2	Metadata	36
10	Performance Optimization	37
10.1	Block Size Selection	37
10.2	Parallel Processing	37
10.3	Memory Management	37
10.4	Performance Tips	37
11	Troubleshooting	38
11.1	Common Errors and Solutions	38
11.1.1	FileNotFoundError	38
11.1.2	MemoryError	38
11.1.3	KeyError	38
11.1.4	ImportError	39
11.1.5	IndentationError	39
11.2	Logging	39
11.3	Debugging Tips	39

12 Support and Community	40
12.1 Citation	40
12.2 Repository	40
12.3 License	40
12.4 Contact	40
A Full Variable Definitions	41
B Configuration File Reference	43
C Example Usage	45
C.1 Running the Engine	45
C.2 Resuming After Interruption	45
C.3 Starting Fresh	45
C.4 Analyzing Results	45
C.5 Example: Custom Configuration	45
D Glossary	47

List of Figures

List of Tables

Listings

Chapter 1

Introduction

1.1 Overview

ClimateProcessingEngine is an open-source Python framework designed for processing large-scale climate datasets, performing statistical distribution fitting, and generating climatological summaries. The engine is optimized for station-wise time series data with support for multiple variables (temperature, precipitation, etc.) and handles missing data through robust window-based approaches.

1.1.1 Key Features

- **Block-oriented processing** – Handles millions of stations efficiently using configurable block sizes
- **Multi-distribution fitting** – Supports Normal, Skew-Normal, Bimodal, and Pearson Type III distributions
- **Parallel processing** – Optional multi-core execution for accelerated computation
- **Checkpoint recovery** – Automatic resumption from the last completed block
- **Zarr output** – Efficient, chunked, cloud-optimized storage format
- **Comprehensive validation** – Data integrity checks at multiple pipeline stages
- **Benchmarking** – Automatic selection of optimal block size based on system resources

1.1.2 Target Audience

- Climate scientists and meteorologists
- Environmental researchers
- Data engineers working with geospatial time series
- Graduate students in climate science

1.1.3 System Requirements

Table 1.1: Minimum System Requirements

Component	Requirement
Operating System	Windows 10/11, Linux, or macOS
Python	3.8 or higher
RAM	16 GB minimum (32 GB recommended)
Storage	100 GB free space (for intermediate and output data)
CPU	4 cores minimum (8+ recommended for parallel mode)

1.1.4 Dependencies

The following Python packages are required:

```

1 numpy>=1.24.0
2 pandas>=2.0.0
3 xarray>=2023.0.0
4 dask>=2023.0.0
5 rasterio>=1.3.0
6 geopandas>=0.14.0
7 cartopy>=0.22.0
8 matplotlib>=3.7.0
9 scipy>=1.10.0
10 scikit-learn>=1.3.0
11 shapely>=2.0.0
12 netcdf4>=1.6.0
13 zarr>=2.15.0
14 numba>=0.57.0
15 pyyaml>=6.0
16 psutil>=5.9.0 (optional, for RAM monitoring)

```

Listing 1.1: Required Dependencies

Chapter 2

Architecture Overview

2.1 System Architecture

The ClimateProcessingEngine follows a modular, pipeline-based architecture with clear separation of concerns. Figure ?? illustrates the overall system design.

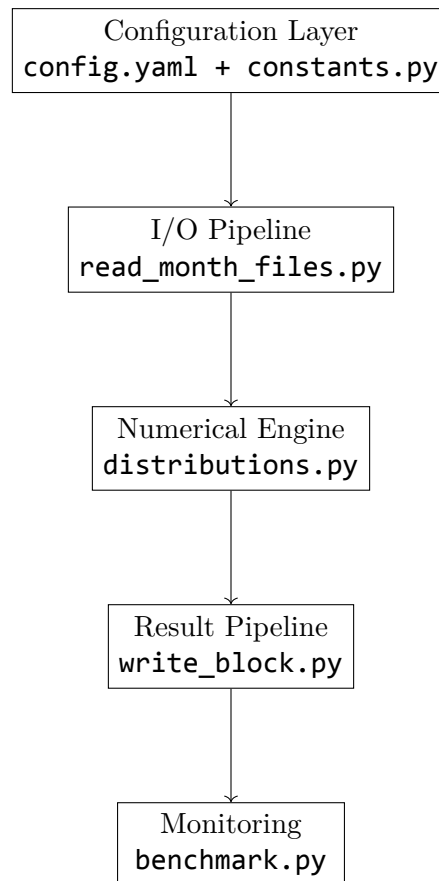


Figure 2.1: System Architecture Overview

2.2 Core Components

2.2.1 Configuration Layer (**constants.py** + **config.yaml**)

Centralizes all settings in a single YAML file, parsed by **constants.py**. This ensures consistent configuration across all modules.

2.2.2 I/O Pipeline (**io_pipeline/*.py**)

Responsible for reading input data from Zarr archives, assembling blocks, and validating loaded data.

2.2.3 Numerical Engine (**numerical_engine/*.py**)

Performs the core statistical analysis, including window extraction, distribution fitting, and result aggregation.

2.2.4 Result Pipeline (**result_pipeline/*.py**)

Validates and writes results to the output Zarr store.

2.2.5 Monitoring (**monitoring/*.py**)

Handles logging, checkpointing, and benchmarking.

2.2.6 Orchestrator (**orchestrator/*.py**)

Coordinates the execution flow, managing the block processing loop.

2.3 Data Flow

The data flow follows a linear pipeline:

1. **Load Phase:** Read raw data from input Zarr files for a block of stations
2. **Assembly Phase:** Arrange data into a 4D array (station, year, day, variable)
3. **Validation Phase:** Validate the assembled data against the data contract
4. **Analysis Phase:** For each station, extract windows and fit distributions
5. **Result Validation:** Validate analysis results before writing
6. **Write Phase:** Write validated results to the output Zarr store
7. **Checkpoint:** Record progress for recovery

Chapter 3

Installation and Setup

3.1 Installation

3.1.1 Clone the Repository

```
1 git clone https://github.com/AminFazlKazemi/ClimateProcessingEngine.git
2 cd ClimateProcessingEngine
```

Listing 3.1: Cloning the Repository

3.1.2 Create a Virtual Environment

```
1 # Windows
2 python -m venv venv
3 venv\Scripts\activate
4
5 # Linux/macOS
6 python3 -m venv venv
7 source venv/bin/activate
```

Listing 3.2: Creating Virtual Environment

3.1.3 Install Dependencies

```
1 pip install -r requirements.txt
```

Listing 3.3: Installing Dependencies

3.2 Configuration

3.2.1 Configuration File Structure

The `config.yaml` file contains all runtime settings:

```
1 # Paths
2 paths:
3   calendar_file: K:/Temp/needed/calendar.txt
4   checkpoint_file: checkpoint.csv
```

```
5  output_dir: I:/climatology_366_rolling
6  output_zarr_name: climatology_stationwise_final.zarr
7  zarr_base: K:/gozareshha/dr vazife/140504 - qc temp/zarr_yearly_monthly
8
9  # Time Range
10 years:
11     start: 1369
12     end: 1399
13 days: 366
14 window_days: 2
15
16 # Processing
17 block_size: 1000
18 cores: 6
19 use_parallel: false
20
21 # Variables
22 variables:
23 - tmin
24 - tmean
25 - tmax
26 fit_variable_index: 1
27
28 # Validation
29 validation:
30     validate_after_load: true
31     validate_before_write: true
32     validate_every_n_blocks: 5
33
34 # Logging
35 logging:
36     level: INFO
37     log_file: climatology.log
38     log_timestamps: true
39
40 # Benchmark
41 benchmark:
42     enabled: true
43     run_on_first_block: true
44     test_block_sizes:
45     - 500
46     - 1000
47     - 1500
48     - 2000
49     - 3000
```

Listing 3.4: Example config.yaml

3.2.2 Understanding Configuration Parameters

Paths

Table 3.1: Path Configuration Parameters

Parameter	Description
<code>calendar_file</code>	Path to the Persian calendar mapping file
<code>checkpoint_file</code>	Location for storing recovery checkpoint
<code>output_dir</code>	Directory where output Zarr will be saved
<code>output_zarr_name</code>	Name of the output Zarr file
<code>zarr_base</code>	Directory containing input Zarr files (organized as <code>YYYY_MM.zarr</code>)

Time Configuration

Table 3.2: Time Configuration Parameters

Parameter	Description
<code>years.start</code>	First year of the analysis (Persian calendar)
<code>years.end</code>	Last year of the analysis (inclusive)
<code>days</code>	Number of days in the year (365 or 366)
<code>window_days</code>	Days before/after each target day for window extraction

The window size is calculated as: `WINDOW_SIZE = 2 * window_days + 1`

Processing Configuration

Table 3.3: Processing Configuration Parameters

Parameter	Description
<code>block_size</code>	Number of stations per processing block
<code>cores</code>	Number of CPU cores to use when parallel processing is enabled
<code>use_parallel</code>	If true, enables multi-core processing

3.3 Preparing Input Data

3.3.1 Input Zarr Structure

The engine expects input Zarr files organized by year and month:

```
1 zarr_base/  
2 --- 1369_01.zarr/  
3 |   --- tmin (2D: day, point)  
4 |   --- tmax (2D: day, point)  
5 |   --- tmean (2D: day, point)  
6 |   --- stationid (1D: point)  
7 |   --- lat (1D: point)  
8 |   --- lon (1D: point)  
9 |   --- elev (1D: point)  
10 --- 1369_02.zarr/  
11 --- ...
```

Listing 3.5: Input Zarr Directory Structure

3.3.2 Calendar File Format

The calendar file (**calendar.txt**) maps Persian dates to day-of-year indices:

```
1 ShamsiDate  ShamsiDOY  GregorianDate  GregorianDOY  
2 13690101    1         19900322      80  
3 13690102    2         19900323      81  
4 ...
```

Listing 3.6: calendar.txt Format

Chapter 4

Core Modules Reference

4.1 Configuration Module (`constants.py`)

4.1.1 Overview

The `constants.py` module loads configuration from `config.yaml` and defines all constants used throughout the engine.

4.1.2 Key Constants

```
1 # Time
2 YEAR_START = 1369
3 YEAR_END = 1399
4 N_YEARS = 31
5 N_DAYS = 366
6 WINDOW_DAYS = 2
7 WINDOW_SIZE = 5                # 2*window_days + 1
8
9 # Variables
10 VARS = ['tmin', 'tmean', 'tmax']
11 N_VARS = 3
12 FIT_VARS = ['tmin', 'tmean', 'tmax']
13 N_OUTPUTS = 90                # 30 outputs per variable x 3 variables
14
15 # Paths
16 ZARR_BASE = "...
17 OUTPUT_DIR = "...
18 OUTPUT_ZARR = "...
19
20 # Processing
21 BLOCK_SIZE = 1000
22 N_CORES = 6
23 USE_PARALLEL = False
24
25 # Validation thresholds
26 MAX_VALUES_PER_FIT = 155      # N_YEARS * WINDOW_SIZE
27 MIN_VALID_VALUES = 5
28 VALID_BEST_DIST = {-1, 0, 1, 2, 3}
29
30 # Data types
31 FLOAT_DTYPE = np.float32
```

```
32 INT_DTYPE = np.int32
```

Listing 4.1: Constants Defined

4.1.3 Configuration Functions

```
1 def load_config(path):
2     """Load YAML configuration file"""
3     with open(path, 'r', encoding='utf-8') as f:
4         return yaml.safe_load(f)
5
6 def ensure_directories():
7     """Create output directories if they don't exist"""
8     os.makedirs(OUTPUT_DIR, exist_ok=True)
9
10 def validate_config():
11     """Validate configuration values and raise errors on invalid settings
12     """
13     # Checks for valid year range, block size, etc.
```

Listing 4.2: Configuration Functions

4.2 Zarr Schema Module (`zarr_schema.py`)

4.2.1 Overview

The `zarr_schema.py` module defines the output schema for the Zarr store. It serves as the single source of truth for all output variables.

4.2.2 Variable Definitions

Each output variable is defined as a tuple: `(name, dtype, fill_value, description)`.

Table 4.1: Output Variables per Temperature Variable

Category	Variables	Count
Best Distribution	<code>best_dist</code>	1
Statistics	<code>mean, std, skewness, median, count</code>	5
Normal Parameters	<code>normal_p1, p2, loglik, aicc, bic</code>	5
Skew-Normal Parameters	<code>skew_p1, p2, p3, loglik, aicc, bic</code>	6
Bimodal Parameters	<code>bimodal_p1, p2, p3, p4, p5, loglik, aicc, bic</code>	8
Pearson III Parameters	<code>pearson_p1, p2, p3, loglik, aicc</code>	5
Total per variable		30
Total for 3 variables		90

Note: The actual implementation uses 30 outputs per variable (excluding `pearson_bic`), resulting in 87 total variables.

4.2.3 Key Functions

```

1 def create_zarr_store(output_path, n_stations, chunk_size=(366, 100)):
2     """
3     Create a Zarr store with all output variables.
4
5     Parameters:
6         output_path: str - Path to create the Zarr store
7         n_stations: int - Total number of stations
8         chunk_size: tuple - Chunk size for Zarr arrays (day, station)
9
10    Returns:
11        zarr.Group: The root group of the Zarr store
12    """
13    # Creates arrays for each variable with specified dtype and fill value
14
15 def add_coords_and_metadata(ds, station_ids, lons, lats, elevs):
16     """
17     Add coordinate variables and metadata to an xarray Dataset.
18
19     Parameters:
20         ds: xarray.Dataset - Dataset to augment
21         station_ids: array - Station identifiers
22         lons, lats, elevs: arrays - Geographic metadata
23
24     Returns:
25         xarray.Dataset: Dataset with coordinates and metadata
26     """
27
28 def create_empty_block_result(block_size):
29     """
30     Create an empty block_result dictionary.
31
32     Parameters:
33         block_size: int - Number of stations in the block
34
35     Returns:
36         dict: Empty result container with all variables initialized
37     """

```

Listing 4.3: Zarr Schema Functions

4.3 Calendar Tables Module (**calendar_tables.py**)

4.3.1 Overview

Provides calendar conversion functions for the Persian calendar system.

4.3.2 Key Functions

```

1 def is_persian_leap_year(year):
2     """Check if a Persian year is a leap year."""
3
4 def get_persian_month_days(month, year):
5     """Get the number of days in a Persian month."""

```

```

6
7 def get_shamsi_doy(year, month, day):
8     """Calculate the day-of-year for a Persian date."""
9
10 def build_doy_table():
11     """Build the day-of-year lookup table."""
12
13 def build_calendar_tables():
14     """Build all calendar tables, using calendar.txt if available."""

```

Listing 4.4: Calendar Functions

4.4 Runtime Tables Module (**runtime_tables.py**)

4.4.1 Overview

Builds tables needed at runtime, independent of the calendar system.

4.4.2 Key Functions

```

1 def build_window_table():
2     """Build the window table for all days."""
3     # For each day i, returns [i-2, i-1, i, i+1, i+2] with wrapping at
4     # boundaries
5
6 def build_file_map(zarr_base):
7     """Build a map from (year, month) to Zarr file path."""
8
9 def build_runtime_tables(zarr_base):
10     """Build all runtime tables."""

```

Listing 4.5: Runtime Tables Functions

4.5 I/O Pipeline Module (**io_pipeline/*.py**)

4.5.1 read_month_files.py

```

1 def read_month_files(block_start, block_size, file_map, year_list):
2     """
3     Read data from Zarr files for a block of stations.
4
5     Parameters:
6         block_start: int - Starting station index
7         block_size: int - Number of stations to read
8         file_map: dict - Mapping from (year, month) to file path
9         year_list: list - All years in the analysis period
10
11     Returns:
12         dict: {(year_idx, month, var_idx): ndarray}
13     """
14     # For each year and month, opens the corresponding Zarr file
15     # Slices the required station range

```

```

16 # Extracts temperature variables and applies scaling for tmean
17 # Returns a dictionary of raw arrays

```

Listing 4.6: read_month_files.py

4.5.2 assemble_block.py

```

1 def assemble_block(data_dict, doy_table, block_size, year_list):
2     """
3     Assemble raw data into a 4D array.
4
5     Parameters:
6         data_dict: dict - Raw data from read_month_files
7         doy_table: ndarray - Day-of-year lookup table
8         block_size: int - Number of stations
9         year_list: list - All years
10
11     Returns:
12         ndarray: Shape (block_size, N_YEARS, N_DAYS, N_VARS)
13     """
14     # Initializes array with NaN values
15     # For each year, month, and variable:
16     #     Maps days to correct day-of-year positions
17     #     Copies data to the assembled array
18     # Missing days remain as NaN

```

Listing 4.7: assemble_block.py

4.5.3 validate_block.py

```

1 def validate_block(block_data, block_start, block_size, strict=True):
2     """
3     Validate assembled block data against the data contract.
4
5     Parameters:
6         block_data: ndarray - The assembled block
7         block_start: int - Starting station index
8         block_size: int - Number of stations
9         strict: bool - Whether to raise exceptions on validation failure
10
11     Returns:
12         dict: Validation report with status, warnings, and errors
13     """
14     # Checks:
15     # - Shape correctness
16     # - Data type
17     # - Contiguous memory layout
18     # - NaN ratio
19     # - Infinite values
20     # - Per-station data completeness

```

Listing 4.8: validate_block.py

4.6 Numerical Engine Module (numerical_engine/*.py)

4.6.1 window_engine.py

```

1 def extract_window_values_fast(station_data, window_table, var_idx):
2     """
3     Extract window values for all days of a station.
4
5     Parameters:
6         station_data: ndarray - Shape (N_YEARS, N_DAYS, N_VARS)
7         window_table: list - Window indices for each day
8         var_idx: int - Variable index to extract
9
10    Returns:
11        list: 366 entries, each containing clean values array or None
12    """
13    # For each day:
14    #     Extracts values from window days
15    #     Removes NaN values
16    #     Returns array if length >= MIN_VALID_VALUES, else None

```

Listing 4.9: window_engine.py

4.6.2 distributions.py

This module implements the core distribution fitting functions using Numba JIT compilation.

Normal Distribution

```

1 @njit
2 def fit_normal_full_numba(data):
3     """
4     Fit a Normal distribution to data.
5
6     Returns:
7         tuple: (mean, std, loglikelihood, AICc, BIC, n_parameters)
8     """
9     # Calculates mean and standard deviation
10    # Computes log-likelihood
11    # Calculates AICc and BIC

```

Listing 4.10: Normal Distribution Fitting

Skew-Normal Distribution

```

1 @njit
2 def skewness_to_alpha(skew):
3     """Convert sample skewness to skew-normal shape parameter alpha"""
4
5 @njit
6 def fit_skewnorm_full_numba(data):
7     """
8     Fit a Skew-Normal distribution using method of moments.

```

```

9
10     Returns:
11         tuple: (alpha, loc, scale, loglikelihood, AICc, BIC, n_parameters)
12     """
13     # Estimates initial parameters from moments
14     # Computes log-likelihood
15     # Calculates AICc and BIC

```

Listing 4.11: Skew-Normal Fitting

Bimodal Normal Distribution

```

1 @njit
2 def fit_bimodal_full_numba(data):
3     """
4     Fit a Bimodal Normal distribution using moment-based estimation.
5
6     Returns:
7         tuple: (w1, mu1, sigma1, mu2, sigma2, loglikelihood, AICc, BIC,
8             n_parameters)
9     """
10    # Estimates 5 parameters from moments:
11    #   w1: mixing weight of component 1
12    #   mu1, sigma1: mean and std of component 1
13    #   mu2, sigma2: mean and std of component 2
14    # Uses kurtosis to estimate separation between modes

```

Listing 4.12: Bimodal Fitting

Pearson Type III Distribution

```

1 @njit
2 def fit_pearson3_full_numba(data):
3     """
4     Fit a Pearson Type III (Gamma) distribution.
5
6     Returns:
7         tuple: (shape, scale, location, loglikelihood, AICc, BIC,
8             n_parameters)
9     """
10    # Uses method of moments with skewness adjustment
11    # Applies Wilson-Hilferty transformation for initial estimates
12    # Computes log-likelihood via Gamma PDF

```

Listing 4.13: Pearson Type III Fitting

Master Fitting Function

```

1 @njit
2 def fit_all_distributions_numba(data):
3     """
4     Fit all four distributions and select the best.
5
6     Returns:
7         ndarray: 30-element array containing:

```

```

8         [0] best_dist (0=Normal, 1=Skew, 2=Bimodal, 3=Pearson, -1=
failed)
9         [1] mean
10        [2] std
11        [3] skewness
12        [4] median
13        [5] count
14        [6] normal_p1 (mean)
15        [7] normal_p2 (std)
16        [8] normal_loglik
17        [9] normal_aicc
18        [10] normal_bic
19        [11] skew_p1 (alpha)
20        [12] skew_p2 (loc)
21        [13] skew_p3 (scale)
22        [14] skew_loglik
23        [15] skew_aicc
24        [16] skew_bic
25        [17] bimodal_p1 (w1)
26        [18] bimodal_p2 (mu1)
27        [19] bimodal_p3 (sigma1)
28        [20] bimodal_p4 (mu2)
29        [21] bimodal_p5 (sigma2)
30        [22] bimodal_loglik
31        [23] bimodal_aicc
32        [24] bimodal_bic
33        [25] pearson_p1 (shape)
34        [26] pearson_p2 (scale)
35        [27] pearson_p3 (location)
36        [28] pearson_loglik
37        [29] pearson_aicc
38        """
39        # Fits all 4 distributions
40        # Selects best using AICc
41        # Returns 30-element array with all parameters

```

Listing 4.14: fit_all_distributions_numba

4.6.3 analyze_station.py

```

1 def analyze_station(station_data, window_table, var_indices):
2     """
3     Analyze a single station for multiple variables.
4
5     Parameters:
6         station_data: ndarray - Shape (N_YEARS, N_DAYS, N_VARS)
7         window_table: list - Window indices
8         var_indices: list - Variable indices to analyze
9
10    Returns:
11        dict: Result dictionary with 87 keys (29 per variable)
12    """
13    # For each variable index:
14    #     Extracts window values using extract_window_values_fast
15    #     For each day:
16    #         Fits distributions using fit_distribution

```



```
17 # Populates result dictionary with parameters
```

Listing 4.15: analyze_station.py

4.6.4 merge_results.py

```
1 def merge_station_result(block_result, station_result, local_idx):
2     """Merge a station's results into the block result."""
3
4 def create_and_merge_results(block_data, window_table, var_indices):
5     """
6     Process all stations in a block and merge results.
7
8     Returns:
9         dict: Complete block result with 87 variables
10    """
11    # Iterates through stations
12    # Calls analyze_station for each
13    # Merges results into block_result
```

Listing 4.16: merge_results.py

4.7 Result Pipeline Module (result_pipeline/*.py)

4.7.1 validate_result.py

```
1 def validate_result(block_result, block_start, block_size, strict=True):
2     """
3     Validate analysis results before writing.
4
5     Returns:
6         dict: Validation report
7     """
8     # Checks:
9     # - Shape of each variable
10    # - Data type matches schema
11    # - best_dist contains only valid codes
12    # - count values in valid range
13    # - No negative standard deviations
14    # - No infinite values in loglik/AICc/BIC
```

Listing 4.17: validate_result.py

4.7.2 write_block.py

```
1 def write_block(root, block_result, block_start, block_end):
2     """Write block results to Zarr store."""
3
4 def write_block_safe(root, block_result, block_start, block_end, validate=
5     True):
6     """Write block results with optional validation."""
```

Listing 4.18: write_block.py

4.8 Orchestrator Module (orchestrator/*.py)

4.8.1 process_block.py

```
1 class FitError(Exception): pass
2 class DataError(Exception): pass
3 class IOError(Exception): pass
4
5 def process_block(block_start, block_end, block_idx, file_map, doy_table,
6                  window_table, year_list, root, var_indices,
7                  last_checkpoint_station=None):
8     """
9     Process a single block of stations.
10
11     Returns:
12         dict: Processing statistics for the block
13     """
14     # 1. Load data using read_month_files and assemble_block
15     # 2. Validate loaded data
16     # 3. Analyze using create_and_merge_results
17     # 4. Validate results
18     # 5. Write results
19     # 6. Save checkpoint
20     # 7. Return statistics
```

Listing 4.19: process_block.py

4.9 Monitoring Module (monitoring/*.py)

4.9.1 benchmark.py

```
1 def run_benchmark(load_func, analyze_func, write_func, test_sizes=None):
2     """
3     Run benchmark tests to determine optimal block size.
4
5     Returns:
6         dict: Benchmark results with recommended block size
7     """
8     # For each test size:
9     #   Measures load, analyze, write times
10    #   Monitors RAM usage
11    #   Counts failed fits
12    #   Calculates stations per second
13    # Returns best performing size
14
15 def print_benchmark_report(report):
16     """Pretty-print benchmark results."""
```

Listing 4.20: benchmark.py

4.9.2 checkpoint.py

```
1 def save_checkpoint(block_idx, station_idx, elapsed=None):
2     """Save checkpoint information to file."""
3
4 def load_checkpoint():
5     """Load checkpoint information from file."""
6
7 def delete_checkpoint():
8     """Delete the checkpoint file after successful completion."""
```

Listing 4.21: checkpoint.py

4.9.3 logger.py

```
1 def setup_logger():
2     """Set up logging configuration."""
3
4 logger = setup_logger()
```

Listing 4.22: logger.py

Chapter 5

Data Contract

5.1 Dynamic Distribution Architecture

Starting from version 2.1, the engine supports a **dynamic distribution architecture** that allows adding new distributions without reprocessing the raw data.

5.1.1 Key Concepts

- **Window Cache:** During the initial processing, the engine stores window values (5-day windows for each station and day) in a separate Zarr store (`window_cache.zarr`).
- **Modular Distributions:** Each distribution is defined as a dictionary in `zarr_schema.py` with its parameters, code, and fitting function.
- **Add Distribution Script:** The `add_distribution.py` script reads the window cache, fits a new distribution, and updates the output Zarr with new variables and `best_dist`.

5.1.2 Adding a New Distribution

To add a new distribution (e.g., Logistic):

1. Add the distribution to **DISTRIBUTIONS** in `zarr_schema.py`
2. Implement the fitting function in `distributions.py` and add it to **FIT_FUNCS**
3. Run `python add_distribution.py --dist logistic`

This process does **not** require reading the raw input files again.

5.1.3 Benefits

- No need to reprocess all data for each new distribution
- Saves significant time and computational resources
- Enables experimentation with different distribution families
- Maintains backward compatibility with existing outputs

5.2 Load Phase Contract

5.2.1 Input

- **block_start**: integer – Starting station index
- **block_size**: integer – Number of stations
- **file_map**: dict – (year, month) → file_path
- **year_list**: list – All years in analysis period
- **doy_table**: ndarray – Shape (N_YEARS, 13, 32)

5.2.2 Output

- **block_data**: ndarray – Shape (block_size, N_YEARS, N_DAYS, N_VARS)
- Data type: **float32**
- Memory layout: C-contiguous
- Missing values: **NaN**

5.3 Window Phase Contract

5.3.1 Input

- **station_data**: ndarray – Shape (N_YEARS, N_DAYS, N_VARS)
- **window_table**: list – 366 entries

5.3.2 Output

- List of 366 entries
- Each entry: **ndarray** or **None**
- Non-None arrays contain valid values (NaNs removed)
- Length $\geq$ MIN_VALID_VALUES = 5

5.4 Analyze Phase Contract

5.4.1 Input

- **station_data**: ndarray – Shape (N_YEARS, N_DAYS, N_VARS)

5.4.2 Output

- Dict with 87 keys
- Each key: ndarray – Shape (**N_DAYS**,)
- Types: **int32** for **best_dist** and **count**
- Types: **float32** for all others

5.5 Write Phase Contract

5.5.1 Input

- **block_result**: dict – 87 variables
- Each variable: Shape (**N_DAYS**, **block_size**)

5.5.2 Output

- Zarr store with 87 variables
- Shape: (**N_DAYS**, **n_stations**)
- Coordinated with **day_of_year** and **point** dimensions

5.6 Constants

Table 5.1: Key Constants

Constant	Value	Description
WINDOW_SIZE	5	Days in window (2 before + target + 2 after)
MAX_VALUES_PER_FIT	155	N_YEARS * WINDOW_SIZE
MIN_VALID_VALUES	5	Minimum valid values for fitting
N_OUTPUTS	87	Number of output variables
VALID_BEST_DIST	{-1,0,1,2,3}	Valid distribution codes

5.7 Error Types

Table 5.2: Error Types

Error Type	Description
IOError	Read/write errors (file not found, permission denied)
DataError	Invalid data (shape mismatch, missing values, infinite values)
FitError	Distribution fitting failed (insufficient data, numerical issues)

Chapter 6

Mathematical Background

6.1 Distributions

6.1.1 Normal Distribution

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Parameters: μ (location), $\sigma > 0$ (scale)

6.1.2 Skew-Normal Distribution

$$f(x) = \frac{2}{\omega} \phi\left(\frac{x-\xi}{\omega}\right) \Phi\left(\alpha \frac{x-\xi}{\omega}\right)$$

where ϕ is the standard Normal PDF, Φ the standard Normal CDF, ξ location, $\omega > 0$ scale, α shape.

6.1.3 Bimodal Normal Distribution

$$f(x) = w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) + w_2 \mathcal{N}(x; \mu_2, \sigma_2^2)$$

with $w_1 + w_2 = 1$, $0 \leq w_i \leq 1$, $\sigma_i > 0$.

6.1.4 Pearson Type III Distribution

$$f(x) = \frac{1}{\Gamma(\alpha)\beta^\alpha} (x-\gamma)^{\alpha-1} \exp\left(-\frac{x-\gamma}{\beta}\right)$$

with shape $\alpha > 0$, scale $\beta > 0$, location γ .

6.2 Model Selection Criteria

6.2.1 AIC (Akaike Information Criterion)

$$\text{AIC} = 2k - 2\ln(\hat{L})$$

6.2.2 AICc (Corrected AIC)

$$\text{AICc} = \text{AIC} + \frac{2k(k+1)}{n-k-1}$$

6.2.3 BIC (Bayesian Information Criterion)

$$\text{BIC} = k \ln(n) - 2 \ln(\hat{L})$$

6.3 Bimodality Metrics

6.3.1 Ashman's D

$$D = \frac{|\mu_2 - \mu_1|}{\sqrt{\frac{\sigma_1^2 + \sigma_2^2}{2}}}$$

6.3.2 Bimodality Coefficient

$$\text{BC} = \frac{\text{skewness}^2 + 1}{\text{kurtosis} + 1}$$

6.3.3 Overlap Coefficient

$$\text{OVL} = \int_{-\infty}^{\infty} \min\{f_1(x), f_2(x)\} dx$$

6.4 Window-Based Estimation

For each day d , the engine extracts values from days $d - 2$ to $d + 2$ (total 5 days) across all years:

$$\text{Window}_d = \{x_{y,t} : y \in \text{years}, t \in [d - 2, d + 2]\}$$

with wrapping at boundaries.

Chapter 7

Execution Flow

7.1 Main Function

```
1 def main():
2     # 1. Load configuration
3     # 2. Get station information from input data
4     # 3. Build calendar and runtime tables
5     # 4. Run benchmark (if enabled)
6     # 5. Create Zarr output store
7     # 6. Load checkpoint (if exists)
8     # 7. Process blocks in a loop
9     #     - For each block: load, validate, analyze, write, checkpoint
10    # 8. Finalize Zarr store
11    #     - Add coordinates and metadata
12    #     - Consolidate metadata
13    # 9. Delete checkpoint
14    # 10. Log completion
```

Listing 7.1: Main Execution Flow

7.2 Block Processing Flow

1. Load data for block
2. Validate loaded data
3. Analyze each station
4. Validate results
5. Write results to Zarr
6. Save checkpoint

Figure 7.1: Block Processing Steps

7.3 Recovery Flow

When a failure occurs:

1. On startup, check for checkpoint file
2. If checkpoint exists, load the last successful block
3. Resume processing from the next station in that block
4. Continue with the main processing loop

Chapter 8

Input Data Format

8.1 Input Zarr Structure

Each monthly Zarr file must contain:

Table 8.1: Input Zarr Variables

Variable	Shape	Description
<code>tmin</code>	<code>(days, points)</code>	Minimum temperature (raw)
<code>tmax</code>	<code>(days, points)</code>	Maximum temperature (raw)
<code>tmean</code>	<code>(days, points)</code>	Mean temperature (raw, scaled by 10)
<code>stationid</code>	<code>(points,)</code>	Station identifiers
<code>lat</code>	<code>(points,)</code>	Latitude
<code>lon</code>	<code>(points,)</code>	Longitude
<code>elev</code>	<code>(points,)</code>	Elevation

8.2 Data Scaling

The `tmean` data is stored with a scaling factor of 10 (actual = stored / 10). The engine automatically handles this scaling during loading.

8.3 Missing Data

Missing values are stored as `NaN` and are handled appropriately.

Chapter 9

Output Format

9.1 Zarr Structure

The output Zarr store contains:

```
1 /
2 --- tmin_best_dist      (day_of_year, point)
3 --- tmin_mean           (day_of_year, point)
4 --- tmin_std            (day_of_year, point)
5 --- tmin_skewness       (day_of_year, point)
6 --- tmin_median         (day_of_year, point)
7 --- tmin_count          (day_of_year, point)
8 --- tmin_normal_p1      (day_of_year, point)
9 --- tmin_normal_p2      (day_of_year, point)
10 --- tmin_normal_loglik  (day_of_year, point)
11 --- tmin_normal_aicc    (day_of_year, point)
12 --- tmin_normal_bic     (day_of_year, point)
13 --- tmin_skew_p1        (day_of_year, point)
14 --- tmin_skew_p2        (day_of_year, point)
15 --- tmin_skew_p3        (day_of_year, point)
16 --- tmin_skew_loglik    (day_of_year, point)
17 --- tmin_skew_aicc      (day_of_year, point)
18 --- tmin_skew_bic       (day_of_year, point)
19 --- tmin_bimodal_p1     (day_of_year, point)
20 --- tmin_bimodal_p2     (day_of_year, point)
21 --- tmin_bimodal_p3     (day_of_year, point)
22 --- tmin_bimodal_p4     (day_of_year, point)
23 --- tmin_bimodal_p5     (day_of_year, point)
24 --- tmin_bimodal_loglik (day_of_year, point)
25 --- tmin_bimodal_aicc   (day_of_year, point)
26 --- tmin_bimodal_bic    (day_of_year, point)
27 --- tmin_pearson_p1     (day_of_year, point)
28 --- tmin_pearson_p2     (day_of_year, point)
29 --- tmin_pearson_p3     (day_of_year, point)
30 --- tmin_pearson_loglik (day_of_year, point)
31 --- tmin_pearson_aicc   (day_of_year, point)
32 --- ... (same for tmean and tmax)
33 --- day_of_year         (day_of_year,)
34 --- point                (point,)
35 --- stationid            (point,)
36 --- lat                  (point,)
37 --- lon                  (point,)
38 --- elev                 (point,)
```

39

--- ...

Listing 9.1: Output Zarr Structure

9.2 Metadata

Global attributes:

Table 9.1: Global Attributes

Attribute	Description
description	"Climatology 366 days - Station-wise architecture"
source	Years range, e.g., "Years 1369-1399, Window ± 2 days"
architecture	"Station-wise, Block-oriented"
version	Engine version
n_outputs	Number of output variables
min_valid_values	Minimum values required for fitting
var_defs	List of all variable definitions

Chapter 10

Performance Optimization

10.1 Block Size Selection

The optimal block size depends on available RAM, CPU cache, disk I/O speed, and number of cores. The benchmarking system automatically evaluates block sizes from 500 to 3000 and selects the one with the highest stations-per-second rate.

10.2 Parallel Processing

When `use_parallel` is enabled, the engine uses `multiprocessing` or `concurrent.futures`; each worker processes a separate block; the `cores` parameter controls the number of workers.

10.3 Memory Management

1. **Block-based processing:** Data is processed in blocks to limit memory usage
2. **Garbage collection:** Active memory is reclaimed after each block
3. **NaN handling:** Missing values use `float32` to reduce memory footprint
4. **Chunked Zarr:** Output is stored in chunks for efficient access

10.4 Performance Tips

1. Use SSD storage for both input and output data
2. Increase the number of cores if available
3. Use smaller block sizes if RAM is limited
4. Disable validation for production runs to reduce overhead
5. Run on a dedicated machine without memory-intensive applications

Chapter 11

Troubleshooting

11.1 Common Errors and Solutions

11.1.1 FileNotFoundError

```
1 FileNotFoundError: [Errno 2] No such file or directory: 'path/to/file'
```

Listing 11.1: FileNotFoundError

Solution:

1. Verify the file path in `config.yaml`
2. Check that the file exists and is readable
3. Use absolute paths to avoid ambiguity

11.1.2 MemoryError

```
1 MemoryError: Unable to allocate array
```

Listing 11.2: MemoryError

Solution:

1. Reduce the `block_size` in `config.yaml`
2. Disable validation for lower memory usage
3. Use `use_parallel: false`
4. Increase system RAM or use a machine with more memory

11.1.3 KeyError

```
1 KeyError: 'variable_name'
```

Listing 11.3: KeyError

Solution:

1. Check that the variable exists in the input Zarr files

2. Verify variable naming matches the schema
3. Ensure the **VARS** list in **config.yaml** is correct

11.1.4 ImportError

```
1 ImportError: No module named 'module_name'
```

Listing 11.4: ImportError

Solution:

1. Install missing dependencies
2. Check Python environment (use **which python**)
3. Use **pip install -r requirements.txt**

11.1.5 IndentationError

```
1 IndentationError: unexpected indent
```

Listing 11.5: IndentationError

Solution:

1. Check that Python files use consistent indentation (4 spaces)
2. Avoid mixing tabs and spaces
3. Use an editor with Python indentation support

11.2 Logging

The engine logs all major events to:

- **Console:** Real-time progress information
- **Log file:** **climatology.log** in the output directory

Log levels: **DEBUG**, **INFO** (default), **WARNING**, **ERROR**.

11.3 Debugging Tips

1. Set **logging.level: DEBUG** for more verbose output
2. Check the log file for detailed error traces
3. Run a small test with **block_size: 100** to verify setup
4. Use Python's interactive debugger (**pdb**) for deep troubleshooting
5. Verify input data quality before large-scale processing

Chapter 12

Support and Community

12.1 Citation

If you use ClimateProcessingEngine in your research, please cite:

```
1 cff-version: 1.2.0
2 title: ClimateProcessingEngine
3 authors:
4   - family-names: Kazemi
5     given-names: Amin Fazl
6 version: v1.0.0
7 date-released: 2026-07-24
8 repository-code: https://github.com/AminFazlKazemi/ClimateProcessingEngine
9 license: MIT
```

Listing 12.1: CITATION.cff

12.2 Repository

<https://github.com/AminFazlKazemi/ClimateProcessingEngine>

12.3 License

MIT License.

12.4 Contact

For questions, issues, or contributions:

- Email: aminfazlkazemi@gmail.com
- GitHub Issues: <https://github.com/AminFazlKazemi/ClimateProcessingEngine/issues>

Appendix A

Full Variable Definitions

Table A.1: Complete Variable Definitions

Variable Name	Dtype	Description
tmin Variables		
tmin_best_dist	int32	Best distribution code (-1=failed, 0=Normal, 1=Skew, 2=Bimodal, 3=Pearson)
tmin_mean	float32	Mean temperature
tmin_std	float32	Standard deviation
tmin_skewness	float32	Skewness
tmin_median	float32	Median
tmin_count	int32	Number of valid values
tmin_normal_p1	float32	Normal: mean
tmin_normal_p2	float32	Normal: standard deviation
tmin_normal_loglik	float32	Normal: log-likelihood
tmin_normal_aicc	float32	Normal: AICc
tmin_normal_bic	float32	Normal: BIC
tmin_skew_p1	float32	Skew-Normal: alpha (shape)
tmin_skew_p2	float32	Skew-Normal: location
tmin_skew_p3	float32	Skew-Normal: scale
tmin_skew_loglik	float32	Skew-Normal: log-likelihood
tmin_skew_aicc	float32	Skew-Normal: AICc
tmin_skew_bic	float32	Skew-Normal: BIC
tmin_bimodal_p1	float32	Bimodal: w1 (mixing weight)
tmin_bimodal_p2	float32	Bimodal: mu1 (mean of component 1)
tmin_bimodal_p3	float32	Bimodal: sigma1 (std of component 1)
tmin_bimodal_p4	float32	Bimodal: mu2 (mean of component 2)
tmin_bimodal_p5	float32	Bimodal: sigma2 (std of component 2)
tmin_bimodal_loglik	float32	Bimodal: log-likelihood
tmin_bimodal_aicc	float32	Bimodal: AICc
tmin_bimodal_bic	float32	Bimodal: BIC
tmin_pearson_p1	float32	Pearson Type III: shape
tmin_pearson_p2	float32	Pearson Type III: scale
tmin_pearson_p3	float32	Pearson Type III: location
tmin_pearson_loglik	float32	Pearson Type III: log-likelihood

Variable Name	Dtype	Description
tmin_pearson_aicc	float32	Pearson Type III: AICc
tmean Variables (same structure)		
<i>... (30 variables with tmean_ prefix)</i>		
tmax Variables (same structure)		
<i>... (30 variables with tmax_ prefix)</i>		

Appendix B

Configuration File Reference

```
1 # Paths
2 paths:
3   calendar_file: K:/Temp/needed/calendar.txt      # Path to Persian
4   calendar_file
5   checkpoint_file: checkpoint.csv                 # Checkpoint file name
6   output_dir: I:/climatology_366_rolling           # Output directory
7   output_zarr_name: climatology_stationwise_final.zarr # Output file name
8   zarr_base: K:/.../zarr_yearly_monthly           # Input Zarr base
9   directory
10
11 # Time Range
12 years:
13   start: 1369                                     # First year (Persian
14   calendar)                                       # Last year (inclusive)
15   end: 1399                                       # Days per year (365 or
16   days: 366                                       # Window size +/- days
17   366)
18 window_days: 2
19
20 # Processing
21 block_size: 1000                                  # Stations per block
22 cores: 6                                           # CPU cores for parallel
23 use_parallel: false                               # Enable parallel
24 processing
25
26 # Variables
27 variables:
28   - tmin                                           # Variables to process
29   - tmean
30   - tmax
31   fit_variable_index: 1                           # Index for fitting (0=
32   tmin, 1=tmean, 2=tmax)
33
34 # Validation
35 validation:
36   validate_after_load: true                       # Validate after loading
37   validate_before_write: true                     # Validate before writing
38   validate_every_n_blocks: 5                     # Validate every N blocks
39
40 # Logging
41 logging:
42   level: INFO                                     # Log level (DEBUG, INFO,
```

```
37     WARNING, ERROR)
38     log_file: climatology.log           # Log file name
39     log_timestamps: true               # Include timestamps in
40     logs
41 # Benchmark
42 benchmark:
43   enabled: true                       # Enable benchmarking
44   run_on_first_block: true           # Run benchmark on first
45   run
46   test_block_sizes:                  # Block sizes to test
47     - 500
48     - 1000
49     - 1500
50     - 2000
51     - 3000
```

Listing B.1: Full config.yaml with Descriptions

Appendix C

Example Usage

C.1 Running the Engine

```
1 cd K:/gozareshha/dr vazife/140504 - qc temp/climatology/climatology_engine
2 python main.py
```

Listing C.1: Basic Execution

C.2 Resuming After Interruption

Simply run `python main.py` again. The engine will automatically detect the checkpoint and resume.

C.3 Starting Fresh

To start from scratch:

```
1 rm checkpoint.csv
2 rm -rf I:/climatology_366_rolling/climatology_stationwise_final.zarr
3 python main.py
```

Listing C.2: Starting Fresh

C.4 Analyzing Results

After the engine completes:

```
1 cd ../../
2 python analyze_distributions1.py
```

Listing C.3: Analyzing Results

C.5 Example: Custom Configuration

```
1 # config.yaml
2 paths:
3   output_dir: /data/climate/output
4   output_zarr_name: my_climate_results.zarr
5   zarr_base: /data/climate/input
6 years:
7   start: 1370
8   end: 1400
9 block_size: 2000
10 cores: 8
11 use_parallel: true
12 variables:
13 - tmin
14 - tmax
```

Listing C.4: Custom Configuration Example

Appendix D

Glossary

Table D.1: Glossary of Terms

Term	Definition
AIC	Akaike Information Criterion – measures model quality balancing fit and complexity.
AICc	Corrected AIC – with small-sample correction.
BIC	Bayesian Information Criterion – alternative to AIC with stronger penalty for complexity.
Block	A contiguous group of stations processed together for efficiency.
Checkpoint	A saved state that allows resuming after interruption.
Doy	Day of Year – a number from 1 to 365 (or 366) representing the day.
Fit	The process of estimating distribution parameters from data.
GEV	Generalized Extreme Value – a family of distributions for extreme events.
Log-likelihood	The logarithm of the likelihood function, used in model selection.
NaN	Not a Number – used to represent missing data.
Numba	A Python library for just-in-time compilation that accelerates numerical computations.
Pearson III	A three-parameter Gamma distribution, also known as a shifted Gamma.
Skew-Normal	A distribution that extends the Normal with a shape parameter for asymmetry.
Window	A set of days around a target day used for estimation (e.g., ± 2 days).
Zarr	A Python library for chunked, compressed, N-dimensional arrays.
Zarr Store	A directory containing Zarr data and metadata.

Appendix E

User Guide

E.1 Getting Started

E.1.1 First Run

To get started with ClimateProcessingEngine, follow these steps:

1. **Clone the repository** as described in Chapter 3.
2. **Install dependencies** using `pip install -r requirements.txt`.
3. **Configure the engine** by editing `config.yaml` (see Section 3.2).
4. **Prepare input data** according to Chapter 8.
5. **Run the engine** using `python main.py`.

E.1.2 Example: Processing a Subset of Data

To test the engine on a small subset of data, modify the configuration as follows:

```
1 # config.yaml (testing)
2 years:
3   start: 1370
4   end: 1375          # Only 6 years for testing
5 block_size: 100      # Small block size for debugging
6 validation:
7   validate_after_load: true
8   validate_before_write: true
9 benchmark:
10  enabled: false      # Skip benchmarking for faster tests
```

Listing E.1: Testing Configuration

E.1.3 Monitoring Progress

During execution, you can monitor progress through:

- **Console output:** Real-time status updates
- **Log file:** `climatology.log` with detailed events
- **Checkpoint file:** `checkpoint.csv` showing completed blocks

E.1.4 Customizing the Configuration

Choosing Variables

The **variables** list determines which climate variables are processed. You can process any subset:

```
1 variables:
2 - tmin
3 - tmax
4 # tmean is excluded
```

Listing E.2: Processing a Subset of Variables

Adjusting the Window Size

The **window_days** parameter controls the window size:

```
1 window_days: 5      # Window size = 2*5+1 = 11 days
```

Listing E.3: Window Size Configuration

Larger windows include more data but may smooth out variability.

Enabling Parallel Processing

For large datasets, enable parallel processing:

```
1 cores: 8
2 use_parallel: true
```

Listing E.4: Parallel Processing Configuration

E.2 Analyzing Results

E.2.1 Loading Output Data

After processing, you can load the results using **xarray**:

```
1 import xarray as xr
2
3 ds = xr.open_zarr('I:/climatology_366_rolling/climatology_stationwise_final
4 .zarr')
5 print(ds)
6
7 # Access a specific variable
8 tmin_mean = ds['tmin_mean'].values
```

Listing E.5: Loading Results

E.2.2 Extracting Best Distributions

To extract the best distribution for each day and station:

```

1 best_dist = ds['tmin_best_dist'].values # Shape (366, n_stations)
2
3 # Distribution codes:
4 # -1 = failed
5 # 0 = Normal
6 # 1 = Skew-Normal
7 # 2 = Bimodal
8 # 3 = Pearson Type III

```

Listing E.6: Extracting Best Distributions

E.2.3 Visualizing Results

Distribution Map

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Count distribution types
5 codes, counts = np.unique(best_dist, return_counts=True)
6 labels = ['Failed', 'Normal', 'Skew-Normal', 'Bimodal', 'Pearson']
7
8 plt.figure(figsize=(10, 6))
9 plt.bar(labels, counts)
10 plt.title('Distribution of Best Fit Types')
11 plt.xlabel('Distribution')
12 plt.ylabel('Number of Stations')
13 plt.show()

```

Listing E.7: Creating a Distribution Map

Temperature Mean Map

```

1 import cartopy.crs as ccrs
2 import cartopy.feature as cfeature
3
4 fig = plt.figure(figsize=(12, 8))
5 ax = plt.axes(projection=ccrs.PlateCarree())
6 ax.add_feature(cfeature.COASTLINE)
7 ax.add_feature(cfeature.BORDERS)
8
9 # Plot the mean temperature for a specific day (e.g., day 180)
10 day_idx = 180
11 sc = ax.scatter(ds['lon'], ds['lat'], c=ds['tmin_mean'][day_idx],
12               cmap='coolwarm', s=5, transform=ccrs.PlateCarree())
13 plt.colorbar(sc, label='Temperature (°C)')
14 plt.title('Mean Temperature - Day 180')
15 plt.show()

```

Listing E.8: Plotting Mean Temperature

E.2.4 Exporting Data

Export to NetCDF

```
1 ds.to_netcdf('climatology_results.nc')
```

Listing E.9: Exporting to NetCDF

Export to CSV

```
1 import pandas as pd
2
3 # Convert to DataFrame for a specific day
4 df = ds['tmin_mean'].sel(day_of_year=180).to_dataframe()
5 df.to_csv('tmin_mean_day180.csv')
```

Listing E.10: Exporting to CSV

Appendix F

Advanced Topics

F.1 Parallel Processing in Depth

F.1.1 How Parallel Processing Works

When `use_parallel` is enabled, the engine distributes blocks across worker processes:

1. The orchestrator divides the stations into blocks.
2. Each block is assigned to a worker process.
3. Workers process blocks independently.
4. Results are written to the Zarr store synchronously.

F.1.2 Performance Considerations

- **CPU-bound:** Use parallel processing for CPU-intensive fitting.
- **I/O-bound:** For I/O-intensive workloads, parallel processing may not improve performance.
- **Memory:** Each worker uses its own memory space; ensure sufficient RAM.

F.1.3 Tuning for Your System

The optimal number of cores depends on your system:

```
1 # For systems with hyperthreading
2 cores = min(cpu_count(), 8) # Cap at 8 for efficiency
3
4 # For systems with limited RAM
5 cores = cpu_count() // 2 # Use half the cores to reduce memory pressure
```

Listing F.1: Choosing the Number of Cores

F.2 Memory Management Strategies

F.2.1 Understanding Memory Usage

The engine's memory usage is primarily determined by:

1. **Block size:** Larger blocks use more memory.
2. **Number of variables:** More variables increase memory usage.
3. **Parallel workers:** Each worker has its own memory footprint.

F.2.2 Estimating Memory Requirements

To estimate memory usage:

$$\text{Memory (GB)} \approx \frac{\text{block_size} \times \text{N_YEARS} \times \text{N_DAYS} \times \text{N_VARS} \times 4}{1024^3}$$

where 4 is the byte size of `float32`.

F.2.3 Reducing Memory Usage

- Use smaller block sizes.
- Process fewer variables at a time.
- Disable validation to save memory.
- Use single-threaded mode (no parallel workers).

F.3 Recovery and Checkpointing

F.3.1 How Checkpointing Works

The engine saves a checkpoint after each block, recording:

- The last processed block index.
- The last station index.
- The elapsed time.

F.3.2 Manual Recovery

If you need to manually resume from a checkpoint:

```

1 # Check the checkpoint file
2 import pandas as pd
3 checkpoint = pd.read_csv('checkpoint.csv')
4 print(checkpoint)
5
6 # Modify the checkpoint to skip failed blocks
7 checkpoint['block_idx'] = 10 # Start from block 10

```

```
8 checkpoint.to_csv('checkpoint_modified.csv', index=False)
```

Listing F.2: Manual Recovery

F.3.3 Troubleshooting Checkpoint Issues

- **Corrupted checkpoint:** Delete the checkpoint file and restart from the beginning.
- **Partial processing:** If a block fails, the checkpoint will not advance.
- **Inconsistent data:** If output data is corrupted, restart from a clean state.

F.4 Adding New Distributions

F.4.1 Dynamic Distribution Architecture

Starting from version 2.1, the engine supports a **dynamic distribution architecture** that allows adding new distributions without reprocessing raw data.

F.4.2 Steps to Add a New Distribution

1. Define the distribution in `zarr_schema.py`:

```
1 DISTRIBUTIONS['logistic'] = {
2     'code': 4,
3     'name': 'Logistic',
4     'params': ['location', 'scale'],
5     'n_params': 2,
6     'fields': ['loglik', 'aicc', 'bic'],
7 }
8
```

Listing F.3: Adding a New Distribution to Schema

2. Implement the fitting function in `distributions.py`:

```
1 from scipy.stats import logistic
2
3 @njit
4 def fit_logistic_numba(data):
5     # Use maximum likelihood estimation
6     # ...
7     return (loc, scale, loglik, aicc, bic)
8
```

Listing F.4: Implementing Logistic Fitting

3. Register the fitting function:

```
1 FIT_FUNCS['logistic'] = fit_logistic_numba
2
```

Listing F.5: Registering the Function

4. Run the add distribution script:

```
1 python add_distribution.py --dist logistic
2
```

Listing F.6: Adding the Distribution

F.4.3 Adding Bimodality Metrics

To add new bimodality metrics, extend the `analyze_station.py` module:

```
1 def compute_harris_d(w1, mu1, sigma1, mu2, sigma2):
2     """Compute Harris's D coefficient."""
3     numerator = abs(mu2 - mu1)
4     denominator = (sigma1 + sigma2) / 2
5     return numerator / denominator if denominator > 0 else 0
```

Listing F.7: Adding a New Bimodality Metric

Appendix G

Frequently Asked Questions

G.1 General Questions

G.1.1 What is ClimateProcessingEngine?

ClimateProcessingEngine is an open-source Python framework for processing large-scale climate datasets, fitting statistical distributions, and generating climatological summaries.

G.1.2 What are the system requirements?

See Table 1.1 for minimum and recommended system requirements.

G.1.3 Is the engine compatible with Windows/Linux/macOS?

Yes, the engine is platform-independent and works on all major operating systems.

G.2 Data Questions

G.2.1 What input data formats are supported?

The engine supports Zarr archives with the structure described in Chapter 8. The data should include `tmin`, `tmax`, `tmean`, and station metadata.

G.2.2 Can I process data without all three variables?

Yes, you can process a subset of variables by modifying the `variables` list in `config.yaml`.

G.2.3 How are missing values handled?

Missing values are represented as `NaN` and are automatically excluded from the analysis. The engine requires at least `MIN_VALID_VALUES` (5) valid values for fitting.

G.3 Performance Questions

G.3.1 How long does processing take?

Processing time depends on:

- Number of stations
- Number of variables
- Block size
- CPU cores
- Disk I/O speed

A benchmark run can estimate the optimal settings for your system.

G.3.2 Can I process data on a laptop?

Yes, but for large datasets, use a machine with sufficient RAM (at least 16 GB). Use smaller block sizes to reduce memory usage.

G.3.3 How can I speed up processing?

1. Use SSD storage for input and output data.
2. Increase the number of cores.
3. Increase the block size (if RAM permits).
4. Disable validation and benchmarking.

G.4 Troubleshooting Questions

G.4.1 Why am I getting a memory error?

Reduce **block_size** in **config.yaml** and/or disable parallel processing. The error indicates insufficient RAM for the current configuration.

G.4.2 Why is the engine not starting?

Check the following:

1. All dependencies are installed.
2. **config.yaml** exists and is correctly formatted.
3. Input data paths are valid.
4. The log file shows detailed error messages.

G.4.3 How do I update to a new version?

Pull the latest changes from the GitHub repository and reinstall dependencies:

```
1 git pull
2 pip install -r requirements.txt --upgrade
```

Listing G.1: Updating to a New Version

Appendix A

Useful Code Snippets

A.1 Post-Processing Analysis

A.1.1 Summarizing Results

```
1 import xarray as xr
2 import numpy as np
3 import pandas as pd
4
5 # Load results
6 ds = xr.open_zarr('climatology_stationwise_final.zarr')
7
8 # Summary statistics
9 print("Dataset overview:")
10 print(ds)
11
12 # Best distribution counts
13 best_dist = ds['tmin_best_dist'].values
14 unique, counts = np.unique(best_dist, return_counts=True)
15 print("\nBest distribution counts:")
16 for code, count in zip(unique, counts):
17     name = ['Failed', 'Normal', 'Skew-Normal', 'Bimodal', 'Pearson'][code
18 +1]
19     print(f" {name}: {count} stations")
```

Listing A.1: Summarizing Results

A.1.2 Extracting Best Model Parameters

```
1 def get_best_parameters(ds, var='tmin', day_idx=0, station_idx=0):
2     """Extract parameters of the best distribution for a specific station
3     and day."""
4     best = ds[f'{var}_best_dist'].values[day_idx, station_idx]
5
6     if best == 0: # Normal
7         return {
8             'mean': ds[f'{var}_normal_p1'].values[day_idx, station_idx],
9             'std': ds[f'{var}_normal_p2'].values[day_idx, station_idx],
10         }
11     elif best == 1: # Skew-Normal
12         return {
```

```

12         'alpha': ds[f'{var}_skew_p1'].values[day_idx, station_idx],
13         'loc': ds[f'{var}_skew_p2'].values[day_idx, station_idx],
14         'scale': ds[f'{var}_skew_p3'].values[day_idx, station_idx],
15     }
16     elif best == 2: # Bimodal
17         return {
18             'w1': ds[f'{var}_bimodal_p1'].values[day_idx, station_idx],
19             'mu1': ds[f'{var}_bimodal_p2'].values[day_idx, station_idx],
20             'sigma1': ds[f'{var}_bimodal_p3'].values[day_idx, station_idx],
21             'mu2': ds[f'{var}_bimodal_p4'].values[day_idx, station_idx],
22             'sigma2': ds[f'{var}_bimodal_p5'].values[day_idx, station_idx],
23         }
24     elif best == 3: # Pearson III
25         return {
26             'shape': ds[f'{var}_pearson_p1'].values[day_idx, station_idx],
27             'scale': ds[f'{var}_pearson_p2'].values[day_idx, station_idx],
28             'loc': ds[f'{var}_pearson_p3'].values[day_idx, station_idx],
29         }
30     else:
31         return None

```

Listing A.2: Extracting Best Model Parameters

A.1.3 Identifying Outliers

```

1 def detect_outliers(ds, var='tmin', threshold=3):
2     """Detect outliers using the mean and standard deviation."""
3     mean = ds[f'{var}_mean'].values
4     std = ds[f'{var}_std'].values
5
6     # Z-score
7     z_scores = np.abs(mean - np.mean(mean)) / std
8
9     # Identify outliers
10    outliers = z_scores > threshold
11
12    return outliers

```

Listing A.3: Identifying Outliers

A.2 Data Integrity Checks

A.2.1 Checking for Missing Values

```

1 def check_missing_values(ds):
2     """Check for missing values in the dataset."""
3     for var in ds.data_vars:
4         missing = np.isnan(ds[var].values).sum()
5         total = ds[var].size
6         print(f"{var}: {missing}/{total} ({100*missing/total:.2f}%)")

```

Listing A.4: Checking for Missing Values

A.2.2 Verifying Distribution Codes

```

1 def verify_distribution_codes(ds, var='tmin'):
2     """Verify that distribution codes are valid."""
3     best = ds[f'{var}_best_dist'].values
4     valid_codes = [-1, 0, 1, 2, 3]
5     invalid = np.isin(best, valid_codes, invert=True)
6     if np.any(invalid):
7         print(f"Invalid codes found: {np.unique(best[invalid])}")
8         return False
9     return True

```

Listing A.5: Verifying Distribution Codes

A.3 Visualization

A.3.1 Heatmap of Best Distributions

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 def plot_distribution_heatmap(ds, var='tmin'):
5     """Plot a heatmap of best distributions for all days."""
6     best = ds[f'{var}_best_dist'].values.T # (n_stations, 366)
7
8     plt.figure(figsize=(15, 10))
9     sns.heatmap(best, cmap='tab10',
10                 xticklabels=50, yticklabels=False,
11                 cbar_kws={'label': 'Distribution Code'})
12     plt.xlabel('Day of Year')
13     plt.ylabel('Station')
14     plt.title(f'Best Distribution for {var}')
15     plt.show()

```

Listing A.6: Heatmap of Best Distributions

A.3.2 Time Series of Parameters

```

1 def plot_parameter_timeseries(ds, var='tmin', param='mean'):
2     """Plot the time series of a parameter."""
3     data = ds[f'{var}_{param}'].values # (366, n_stations)
4
5     plt.figure(figsize=(12, 6))
6     for i in range(min(10, data.shape[1])): # Plot first 10 stations
7         plt.plot(data[:, i], alpha=0.7)
8
9     plt.xlabel('Day of Year')
10    plt.ylabel(f'{param} ({var})')
11    plt.title(f'Time Series of {param} for {var}')
12    plt.grid(True, alpha=0.3)
13    plt.show()

```

Listing A.7: Time Series of Parameters

A.3.3 Comparing Distributions

```
1 def compare_distributions(data, dist_names=['normal', 'skew', 'bimodal', 'pearson']):  
2     """Compare distributions fitted to data."""  
3     # Implementation depends on your analysis needs  
4     pass
```

Listing A.8: Comparing Distributions